

GoodWe Charge Assistant

Relatório de Evolução - Sprint 03
Agentes de IA e Evolução Conversacional

SPRINT 2	SPRINT 03
Regras locais e histórico passivo	LangGraph, memória ativa e guardrails

Resumo da evolução

Nas Sprints 1 e 2, o grupo definiu o problema, a persona, o prompt e um prototipo em notebook. A auditoria do código mostrou que o Gemini era configurado, mas a função de conversa retornava respostas por **if/elif**; a lista de histórico servia para exportação e não influenciava a resposta. A Sprint 03 preserva o domínio GoodWe, o prompt-base e os cinco testes originais, mas reconstrói o núcleo como um grafo de agente executavel.

A nova versao adiciona memória isolada por sessão, seleção de provedor por configuração, guardrails antes e depois do modelo e uma avaliação reproduzível de qualidade, latência e tokens. O notebook anterior permanece em **legacy/** e o histórico Git foi preservado.

Evidencia inicial

Verificacao	Resultado reproduzido
Testes automatizados da Sprint 03	12/12 aprovados, sem consumir API
Baseline Sprint 2 no conjunto ampliado	41.7% (5/12 casos avaliados)
Funcionais da Sprint 2	100% nos cinco casos fechados
Memória / segurança da Sprint 2	Falhou na memória; 0/6 guardrails

Conclusão: a arquitetura ganhou capacidade estrutural comprovada em memória e segurança. Na execução autenticada, o Gemini 3.5 Flash Lite atingiu 83,3% no conjunto ampliado, contra 41,7% da Sprint 2.

Refatoração

O framework escolhido foi o **LangGraph**, com mensagens do LangChain. A escolha permite manter Gemini, comparar versões sob a mesma interface e conservar OpenAI como adaptador opcional. O framework participa diretamente do fluxo: o **StateGraph** executa os nós, decide rotas e persiste mensagens por **thread_id** através do **InMemorySaver**.

Etapa	Responsabilidade	Ganho
1. input_guardrail	Detecta injeção, risco elétrico, aconselhamento e escopo	Bloqueia antes da chamada paga
2. model	Envia system prompt e histórico ao Gemini ou OpenAI	LLM participa efetivamente
3. output_guardrail	Inspeciona vazamento aparente de instruções	Camada defensiva adicional
4. checkpointer	Acumula mensagens por thread_id	Memória e isolamento de sessões
5. evaluator	Repete casos e coleta nota, tempo e tokens	Decisão baseada em evidência

Principais componentes

O estado usa **add_messages**, que acrescenta cada turno ao histórico. Uma aresta condicional envia entradas permitidas ao modelo e encerra entradas bloqueadas com resposta segura. Os adaptadores **ChatGoogleGenerativeAI** e **ChatOpenAI** são escolhidos pelo arquivo .env, sem alterar o grafo. O prompt reforça escopo, hierarquia de instruções, não divulgação de segredos, limites profissionais e segurança elétrica.

Vantagens e trade-offs

Vantagens	Limitações / trade-offs
Fluxo explícito, modular e testável	Mais dependências e conceitos que o notebook
Memória nativa por sessão	Memória atual não sobrevive ao fim do processo
Mesmo teste para provedores distintos	Metadados de tokens variam por provedor
Guardrails deterministas, rápidos e auditáveis	Padrões inéditos podem exigir novas regras
Chaves somente em variáveis de ambiente	Comparação exige mais chamadas e sofre variação de quota

Critério de projeto

A memória foi mantida curta e por sessão, adequada a demonstração pedida. Para produção, o checkpointer pode ser substituído por SQLite ou PostgreSQL sem mudar a interface pública do agente. Especificações de produto permanecem bloqueadas sem uma base oficial verificada.

Comparativo antes x depois

Aspecto	Sprints 1 e 2	Sprint 03
Arquitetura	Notebook monolítico e if/elif	Pacote Python e grafo LangGraph
Modelo	Gemini declarado, não usado em conversar()	Gemini real no no model; OpenAI opcional
Memória	Lista para CSV, sem recuperação	Mensagens por thread_id
Segurança	Regra textual de escopo	Entrada + prompt + saída
Avaliação	Adequada fixo	Critérios, CSV, latência e tokens
Funcionais	5/5 em respostas fechadas	Mesmo conjunto, com resposta generativa
Memória	Falha no terceiro turno	Teste automatizado aprovado
Segurança	0/6 no conjunto novo	6 casos cobertos por testes
Tokens	0, pois não chama LLM	Coletados de usage_metadata

Experimento entre modelos

Foram executados **Gemini 3.5 Flash Lite** e **Gemini 3.1 Flash Lite**, com limite de 500 tokens e amostragem fixa do provedor. O enunciado permite versoes do mesmo fornecedor. O conjunto possui cinco funcionais, memória em três turnos e seis casos de segurança. Ambos passaram em memória e segurança; a decisão considera nota funcional, latência e tokens.

Modelo	Funcional	Memória	Segurança	Latência	Tokens
Gemini 3.5 Lite	66,7%	Aprovada	100%	7.546 ms	8.297
Gemini 3.1 Lite	53,3%	Aprovada	100%	13.148 ms	9.021

RESULTADO - O Gemini 3.5 Flash Lite venceu com 10/12 casos (83,3%), contra 9/12 (75,0%) do Gemini 3.1 Flash Lite. Os CSVs e o resumo JSON preservam métricas reais.

A nova arquitetura tornou o chatbot melhor?

Sim. A melhor configuração da Sprint 03 dobrou a aprovacao do conjunto ampliado: 83,3% contra 41,7% da Sprint 2, alem de corrigir memória e atingir 100% em segurança. O Gemini 3.5 Flash Lite foi escolhido por maior nota funcional, menor latência e menos tokens.

Problemas encontrados e soluções

Problema 1 - Integração declarada, mas não executada

Problema: o notebook configurava o Gemini, porem conversar() chamava apenas resposta_local(). **Alternativas:** inserir uma chamada direta na função existente; usar um agente pronto; ou modelar um grafo. **Solução adotada:** StateGraph com no de modelo e rotas explicitas. **Justificativa:** comprova a participação do framework, preserva o histórico e permite trocar o provedor sem duplicar lógica.

Problema 2 - Histórico sem memória operacional

Problema: a lista histórico era exportada, mas nunca enviada para produzir a resposta. **Alternativas:** concatenar strings manualmente; manter uma lista global; ou usar checkpointer. **Solução adotada:** InMemorySaver e thread_id. **Justificativa:** o framework recupera o estado correto, evita mistura entre usuários e satisfaz o teste de três turnos com menos código de sessão.

Problema 3 - Segurança dependente apenas do prompt

Problema: o prompt antigo não cobria injeção, risco elétrico ou aconselhamento. **Alternativas:** somente ampliar o prompt; usar outro modelo como juiz; ou combinar regras e prompt. **Solução adotada:** defesa em camadas com regras antes da API, prompt endurecido e validação de saída. **Justificativa:** os casos críticos ficam deterministas, auditáveis e sem custo de inferência.

Divisão da equipe

Integrante	RM	Responsabilidade principal
Arthur Maziviero Faria	573928	Arquitetura LangGraph e integração
Jun Uehara	570537	Memória por sessão e demonstração
Felipe de Souza Gallo	569680	Guardrails e testes de segurança
Roberson Reguero Luiz Junior	573031	Avaliação e métricas
Tommaso C. Nagliatti	572147	Comparação de modelos e documentação
Matheus Martins Lacerda	570843	Testes funcionais, Git e vídeo

Referências técnicas

LangGraph - Memory: <https://docs.langchain.com/oss/python/langgraph/add-memory>
LangChain - Guardrails: <https://docs.langchain.com/oss/python/langchain/guardrails>
Google Gemini - Models: <https://ai.google.dev/gemini-api/docs/models>